

Návrh a implemetace databázových systémů

Kursory

Kursory

- dotaz nad databází (např. SELECT) vrací obecně více řádků, tj. tabulku
- mnohdy je třeba zpracovávat vrácené řádky jeden po druhém
- kursor
 - struktura „ukazující“ na řádky výsledku dotazu
 - umožňuje získávat data dotazu postupně po řádcích
 - může předcházet potížím s pamětí, pokud je výsledek dotazu příliš rozsáhlý
 - alternativa k procházení FOR smyčce
 - lze jej použít jako návratovou hodnotu z funkce

Kursory

- deklarace
 - kursorová proměnná typu **refcursor**
 - obecná deklarace kursoru se syntaxí
 - přepínač SCROLL umožní kursoru rolovat nazpátek k předcházejícím řádkům
 - argumenty slouží k dosazování hodnot do dotazu
 - konkrétní hodnoty jsou dodány ve chvíli otevření kursoru
 - slovo IS lze zaměnit s FOR

name [[NO] SCROLL] CURSOR [(arguments)] IS query;

Kursory

- příklady deklarace
 - unbound kursor lze použít pro jakýkoli dotaz
 - bound kursory už mají svůj dotaz přímo přidělen

DECLARE

 curs1 refcursor; -- nevázaný (unbound) kursor

 curs2 CURSOR FOR SELECT * FROM t1;

 curs3 CURSOR(key integer) IS SELECT * FROM t1

WHERE unique1 = key;

Kursory

- otevření kursoru
 - unbound kursor
 - kursoru se přiřadí dotaz a otevře se
 - kursor musí být deklarován jako unbound
 - kursor nesmí být již otevřen
 - dotaz musí být něco, co vrací řádky, typicky SELECT
 - v kursoru lze použít proměnnou PL/pgSQL, ale její hodnota se dosadí POUZE při otevření kursoru; její další změny nemají na funkci kursoru již vliv

```
OPEN curs1 NO SCROLL FOR SELECT * FROM foo  
WHERE key = mykey;
```

Kursory

- otevření kursoru
 - bound kursor
 - předají se případné parametry a otevře se
 - SCROLL / NO SCROLL už je dáno deklarací
 - kursor nesmí být již otevřen
 - dotaz musí být něco, co vrací řádky, typicky SELECT
 - v kursoru lze použít proměnnou PL/pgSQL, ale její hodnota se dosadí POUZE při otevření kursoru; její další změny nemají na funkci kursoru již vliv

```
OPEN curs2;  
OPEN curs3(42);
```

Kursory

- použití kursoru
 - pro použití musí být kursor otevřen, i když ne nutně v té samé funkci
 - kursor je použitelný v rámci aktuální transakce, pak se zavře

```
FETCH curs1 INTO rowvar;  
FETCH curs2 INTO foo, bar, baz;  
FETCH LAST FROM curs3 INTO x, y;  
FETCH RELATIVE -2 FROM curs4 INTO x;
```

Kursory

- k získání dat slouží příkaz **FETCH**
 - vloží další řádek kursoru do targetu
 - není-li další řádek, vloží tam hodnoty NULL
 - někdy nefunguje dobře; pro kontrolu, zda byl řádek vložen je lepší kontrolovat proměnnou **FOUND**
 - klauzule direction
 - **NEXT** (nebo **FORWARD**) - další řádek
 - **PRIOR** (nebo **BACKWARD**) - předchozí řádek
 - **FIRST** - první řádek
 - **LAST** - poslední řádek
 - **ABSOLUTE n** - n-tý řádek dotazu absolutně
 - **RELATIVE n** - n-tý řádek dotazu relativně vůči akt.pozici
 - pokud n=0, vrací aktuální řádek bez posunu polohy
 - bez klauzule direction se předpokládá **NEXT**
- FETCH [direction FROM] cursor INTO target;**

Kursory

- příkaz MOVE
 - přesune kursor bez vracení dat
 - úplně stejné možnosti jako standardní FETCH
 - bez specifikace *direction* se bere NEXT

MOVE [direction FROM] cursor;

MOVE curs1;

MOVE LAST FROM curs3;

MOVE RELATIVE -2 FROM curs4;

Kursory

- aktualizace dat pod kursorem
 - lze mazat či aktualizovat data v tabulce, kam ukazuje kurzor
 - dotaz pod kursorem musí být pro tuto operaci vhodný, tj. žádné GROUP BY apod.
 - pokud chceme tuto možnost používat, je nejlepší definovat *bound* kurzor jako FOR UPDATE

UPDATE table SET ... WHERE CURRENT OF cursor;
DELETE FROM table WHERE CURRENT OF cursor;

UPDATE foo SET dataval = myval WHERE CURRENT OF
curs1;

Kursory

- uzavření kursoru
 - kursor lze uzavřít a tím uvolnit zdroje dříve, než skončí aktuální transakce

`CLOSE cursor;`

Kursory

- kursor jako návratová hodnota funkce
- kursor lze vracet z funkce

```
CREATE TABLE test (col text);  
INSERT INTO test VALUES ('123');
```

```
CREATE FUNCTION reffunc(refcursor) RETURNS refcursor AS $$  
BEGIN  
    OPEN $1 FOR SELECT col FROM test;  
    RETURN $1;  
END; $$ LANGUAGE plpgsql;
```

```
BEGIN; -- začátek transakce  
    SELECT reffunc('funcursor');  
    FETCH ALL IN funcursor;  
COMMIT;
```

Kursory

- kursor ve smyčce
 - speciální varianta FOR cyklu
 - otevře
 - procykluje
 - uzavře při opuštění smyčky
 - kursor nesmí být předtím otevřen

FOR recordvar IN bound_cursorvar [(argument_values)] LOOP
 statements

END LOOP;

Kursory

- kursor s dynamickým příkazem
 - chceme-li například kursoru přiřadit pokaždé trochu jiný dotaz, použijeme analogii příkazu EXECUTE
 - namísto

OPEN curs FOR SELECT * FROM mytable;

- použijeme například

OPEN curs FOR EXECUTE 'SELECT * FROM ' || quote_ident(\$1)